

Software Architecture Document (SAD)

Smart Complaint Management System

Field	Detail
Project	Smart Complaint Management System (SCMS)
Team	Team 8
Version	1.0
Authors	Team 8 — Aryaman D K, Vikas Gowda L, V S R Vinay Varma, Shridhar Umesh Golipalle
Team Members (SRN)	PES1UG24AM344 PES1UG24AM329 PES1UG24AM317 PES1UG25AM814
Date	30-09-2026
Status	Draft

Revision History

Version	Date	Author	Change Summary	Approval
1.0	30-09-2026	Team 8	Architecture and design with component, sequence diagrams and ADRs	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Smart Complaint Management System (SCMS). It describes how the requirements baselined in the SRS v1.0 are realised: the layers and components of the system, the decisions taken and why, the interactions between components, the public interfaces of the main components, and the security architecture. It is the reference for implementation and for review of the delivered system.

1.2 Scope

Covers the complaint life cycle supported by SCMS: registration and authentication, complaint lodging with evidence, automatic routing to a department, assignment to an agent, SLA-based escalation, status tracking, resolution and closure, notification, reporting and audit. Excluded, as in the SRS: payments, native mobile applications, integration with external grievance systems, and the internal implementation of the e-mail and SMS gateways.

1.3 Audience

Developers of Team 8, QA engineers, security reviewers, the project guide and course coordinator, and any team maintaining the system after handover.

1.4 Definitions

Term	Meaning
SCMS	Smart Complaint Management System
ADR	Architecture Decision Record — a short record of a decision, its context and its consequences
SLA	Service Level Agreement — the maximum permitted resolution time for a complaint category
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper — the Django database abstraction layer
STRIDE	Threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security
DRF	Django REST Framework
Celery / Celery Beat	Asynchronous task worker, and its scheduler for periodic jobs
DPDP Act	Digital Personal Data Protection Act, 2023 (India)

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture: goals, stakeholders, the component diagram, the chosen pattern with its Architecture Decision Records, the technology stack, risks, traceability back to the SRS, and the security architecture including a STRIDE threat model. Section 4 gives the design: sequence diagrams for the two most cross-cutting flows, the API definitions for two components, error handling and monitoring, UX design, and open issues. Readers checking requirement coverage should go to section 3.8; readers reviewing security should go to 3.9.

2.2 Related Documents

Document	Version	Relationship to this document
Software Requirements Specification (SRS)	1.0, 06-09-2026	Source of every requirement ID referenced here; section 3.8 traces components back to it

Document	Version	Relationship to this document
Software Test Plan (STP)	Planned, 02-10-2026	Will take its test cases from the acceptance criteria in SRS section 6 and the interfaces in section 4.3 below
Requirements Traceability Matrix (RTM)	Within SRS section 8	Maps each requirement to a module; this document defines those modules
Jira backlog (project T8)	Live	7 epics, 42 stories and tasks, each labelled with its SRS requirement ID

3. Architecture

3.1 Goals & Constraints

Goals

- Keep complainant data confidential and the complaint life cycle tamper-evident (SRS objectives SO-01 and SO-02).
- Make every state change attributable, so that resolution claims can be audited (SO-02, SO-03).
- Keep the complaint lodging path fast and simple, because the public uses it on mobile browsers (SCMS-NF-001, SCMS-NF-004).
- Keep slow work — notifications and SLA checks — off the request path so the user is never held waiting.
- Stay modular enough that four developers can work on separate areas without colliding (SCMS-NF-005).

Constraints

- Python and Django are fixed by the project assignment; no change of language is permitted.
- Personal data handling must be consistent with the DPDP Act, 2023.
- Deployment budget is 2 vCPU and 4 GB RAM per application instance.
- Only permissively licensed open-source libraries may be used.
- The team is four students working part-time across a single semester, which bounds how much operational complexity the architecture can carry.

3.2 Stakeholders & Concerns

Stakeholder	Primary concern	How the architecture addresses it
Complainant (citizen / guest)	Lodging must be quick and the status must be honest and visible	Notifications and routing run asynchronously, so the lodging request returns as soon as the complaint is stored; TrackingModule serves status from a single state machine
Support Agent	A clear queue and complete context on each complaint	TrackingModule and ComplaintModule expose the complaint, its history and its evidence through one detail view
Department Admin	Workload visibility and no silently missed SLA	Celery Beat drives escalation independently of any user action; ReportModule provides the department dashboard
System Admin	Control of configuration without redeployment, and a trustworthy audit record	AdminModule stores departments, categories and SLAs as data rather than code; AuditModule writes append-only rows
Developers (Team 8)	Four people working in parallel without conflicts	Eight Django apps with distinct responsibilities, matching the module column of the SRS traceability matrix
Course coordinator / evaluator	Demonstrable coverage of the stated requirements	Section 3.8 traces every component to the requirement IDs it implements

3.3 Component (UML) Diagram

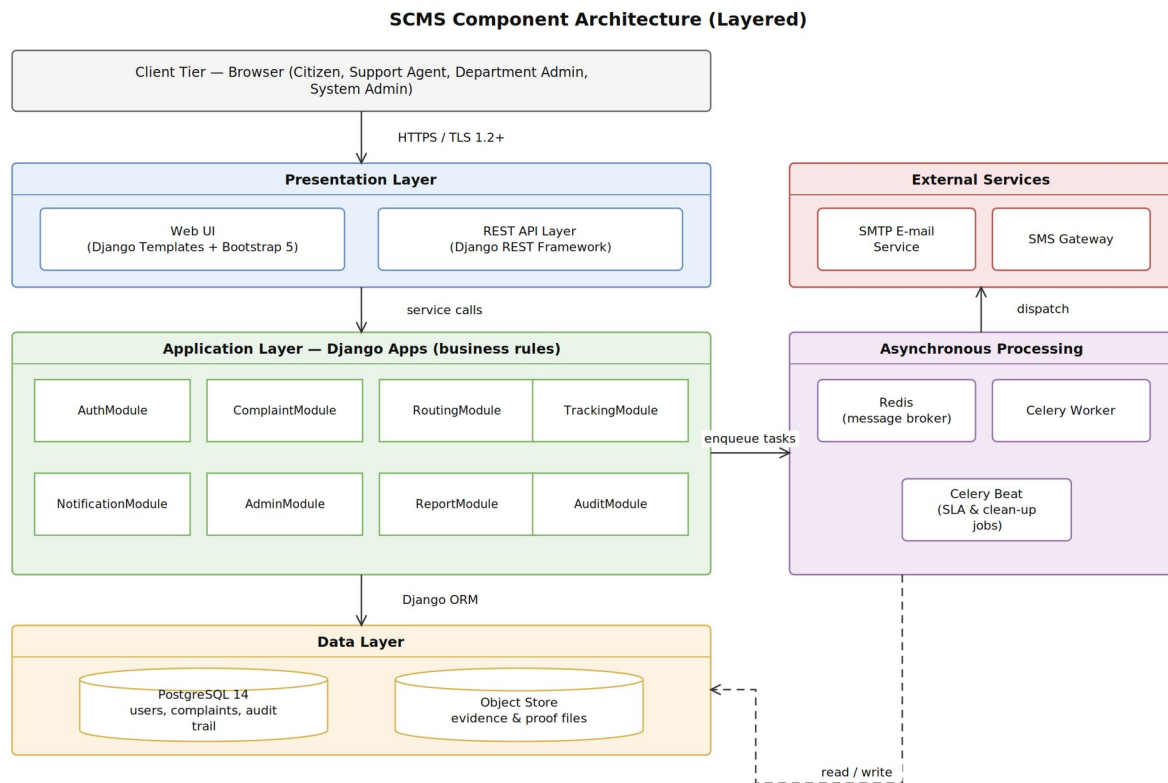


Figure 1: SCMS layered component architecture. Solid arrows are synchronous calls; the dashed arrow is asynchronous access by background workers.

3.4 Component Descriptions

Component	Responsibility	Requirements implemented
Web UI	Renders the role-scoped browser interface using Django templates and Bootstrap 5; performs client-side convenience validation only	SCMS-NF-004
REST API Layer	Exposes the JSON interface used by the browser client and reserved for any future mobile client; handles serialization, authentication and throttling	SCMS-SR-004, SCMS-SR-007
AuthModule	Registration, OTP verification, login, session lifetime, password reset, account lockout and RBAC enforcement	SCMS-F-001 to SCMS-F-005, SCMS-SR-002, SCMS-SR-003
ComplaintModule	Complaint creation and validation, reference ID generation, evidence attachment handling, guest lodging and drafts	SCMS-F-006 to SCMS-F-010, SCMS-SR-005
RoutingModule	Category-to-department routing, agent assignment and reassignment, priority assignment, SLA escalation and duplicate detection	SCMS-F-011 to SCMS-F-015
TrackingModule	Complaint state machine, threaded comments, resolution gating, reopen window and satisfaction rating	SCMS-F-016 to SCMS-F-020
NotificationModule	Composes and dispatches e-mail and SMS messages; applies retry with exponential backoff and records permanent failures	SCMS-F-021, SCMS-F-022
AdminModule	Master data (departments, categories, mapping, SLA durations, priority matrix) and user and role management	SCMS-F-023, SCMS-F-024
ReportModule	Role-scoped dashboards, search and filtering, CSV and PDF export	SCMS-F-025, SCMS-F-026, SCMS-F-028
AuditModule	Writes append-only audit rows for every state change and serves the audit viewer	SCMS-F-027, SCMS-NF-007, SCMS-SR-006

Component	Responsibility	Requirements implemented
Celery Worker	Executes queued tasks: notification dispatch, duplicate detection, export generation	SCMS-F-015, SCMS-F-021, SCMS-F-026
Celery Beat	Time-triggered jobs: SLA breach checks, auto-closure of resolved complaints, draft purge, retention archival	SCMS-F-010, SCMS-F-014, SCMS-F-019, SCMS-NF-007
Redis	Message broker for Celery and store for rate-limit counters	SCMS-SR-007
PostgreSQL	Durable store for users, complaints, comments, attachment metadata, master data and the audit trail	All persistent requirements
Object Store	Holds evidence and resolution-proof files outside the web root, with randomised filenames	SCMS-F-008, SCMS-F-018, SCMS-SR-005

3.5 Chosen Architecture Pattern and Rationale

SCMS uses a layered architecture deployed as a modular monolith: one Django project containing eight applications, one PostgreSQL database, and a separate Celery worker process for asynchronous work. Calls flow downward only — presentation to application to data — and no layer calls back upward.

Microservices were considered and rejected. Splitting eight modules into independently deployed services would add service discovery, inter-service authentication, distributed tracing and eventual-consistency handling, none of which the problem needs at an expected scale of 50,000 complaints per year. It would also create operational work that a four-person student team cannot carry alongside implementation. The modules are already separated by Django app boundaries, so extraction into services later remains possible without rewriting the business logic.

3.5.1 Architecture Decision Records

ID	Decision	Context	Consequences
ADR-01	Layered modular monolith rather than microservices	Four part-time developers, one semester, modest expected load, and no requirement for independent scaling of any single module	Simple deployment and debugging, and one transactional database. The trade-off is that the whole application is deployed as a unit, so a fault in one module can affect the process
ADR-02	PostgreSQL as the single relational store	Complaint, assignment and audit writes must be atomic together; trigram similarity is needed for duplicate detection	Transactional integrity across the life cycle, and pg_trgm available without a second datastore. The trade-off is that unstructured growth would need a schema migration rather than a document store
ADR-03	Asynchronous work via Celery and Redis	Notification dispatch depends on third-party gateways, and SLA checks must run without a user request	The lodging request returns as soon as the complaint is stored, and gateway failure cannot break lodging. The trade-off is two extra processes to run and monitor
ADR-04	Append-only audit table with no UPDATE or DELETE grant	Resolution claims must be defensible during review, and SCMS-SR-006 requires immutability	Tamper evidence is enforced by the database rather than by application discipline. The trade-off is that corrections must be added as new rows rather than edits
ADR-05	Evidence files in an object store outside the web root, not in the database	Attachments are up to 5 MB each and are served back to users	Database size and backup time stay manageable, and uploads cannot be executed as code. The trade-off is that file storage and the database must be backed up consistently with each other
ADR-06	Master data (departments, categories, SLA, priority matrix) stored as rows, not constants	SCMS-F-023 requires configuration changes to take effect without redeployment	Administrators change routing behaviour at runtime. The trade-off is that routing rules must be validated at save time, since a bad mapping would misroute complaints

3.6 Technology Stack & Data Stores

Layer	Technology	Notes
Language / framework	Python 3.11, Django 5.x, Django REST Framework	Fixed by the project assignment
Web server	Gunicorn behind Nginx	Nginx terminates TLS and serves static assets
Presentation	Django templates, Bootstrap 5	Responsive from 320 px; no separate front-end build step
Database	PostgreSQL 14+	pg_trgm extension for duplicate-complaint similarity
Asynchronous tasks	Celery with Redis broker, Celery Beat scheduler	Notification dispatch, SLA escalation, auto-closure, draft purge
File storage	Local filesystem or S3-compatible object store	Outside the web root; randomised filenames
External services	SMTP relay, HTTP SMS gateway	Credentials held in environment variables, never in source

Layer	Technology	Notes
Deployment	Docker containers on Linux (Ubuntu 22.04 LTS)	Same image for application and worker, different endpoints
Testing / quality	pytest with coverage, flake8, OWASP ZAP, JMeter	Supports SCMS-NF-005, SCMS-NF-001 and the security suite

3.7 Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
SMS or e-mail gateway unavailable, so complainants are not notified	Medium	Medium	Dispatch runs asynchronously with three retries and exponential backoff; permanent failures are logged for administrator review (SCMS-F-022)
Celery Beat stops, so SLA escalations silently never fire	Low	High	Beat heartbeat monitored; dashboard surfaces complaints past deadline so a stalled scheduler becomes visible to Department Admins
Spam or flooding through the public guest complaint form	High	Medium	Rate limiting of 5 submissions per mobile number per hour, CAPTCHA on guest lodging, and duplicate detection (SCMS-SR-007, SCMS-F-015)
Malicious file uploaded as evidence	Medium	High	MIME and extension validation, malware scan, storage outside the web root with randomised names, served as an attachment (SCMS-SR-005)
A user reaching another user's complaint by changing the URL	Medium	High	Deny-by-default authorisation with an object-level ownership check on every complaint operation (SCMS-SR-003)
Database growth degrades dashboard and search response time	Medium	Medium	Indexes on status, department, category and created date; three-year retention with archival (SCMS-NF-007)
Single deployed unit means one fault can affect the whole application	Medium	Medium	Accepted consequence of ADR-01; mitigated by module boundaries, unit test coverage of at least 70%, and running two application instances behind the load balancer
Team member unavailable during the semester	Medium	Medium	Module ownership documented in the team roles table, with code reviewed across pairs so no module has only one informed developer

3.8 Traceability to Requirements

Each component below implements the listed SRS requirements. The mapping matches the Module column of the RTM in SRS section 8.

Requirement group (SRS)	Component	Realised by
SCMS-F-001 to SCMS-F-005 (authentication, roles)	AuthModule	Django authentication with Argon2id hashing, OTP via SMS gateway, RBAC decorators and object-level permission checks
SCMS-F-006 to SCMS-F-010 (lodging, evidence)	ComplaintModule	Complaint model and form validation, reference ID sequence, attachment handler writing to the object store
SCMS-F-011 to SCMS-F-015 (routing, escalation)	RoutingModule	Category-to-department mapping lookup, assignment service, priority matrix, Celery Beat SLA job, pg_trgm similarity check
SCMS-F-016 to SCMS-F-020 (tracking, resolution)	TrackingModule	State machine with validated transitions, comment thread model, resolution gate requiring remarks and proof, reopen window job
SCMS-F-021, SCMS-F-022 (notification)	NotificationModule + Celery Worker	Task per event with retry policy and a permanent-failure record
SCMS-F-023, SCMS-F-024 (administration)	AdminModule	Master data models and the user and role management screens

Requirement group (SRS)	Component	Realised by
SCMS-F-025, SCMS-F-026, SCMS-F-028 (reporting, search)	ReportModule	Role-scoped dashboard queries, filter set, CSV writer and ReportLab PDF export
SCMS-F-027, SCMS-NF-007 (audit, retention)	AuditModule	Append-only audit model written on every transition, plus the scheduled archival job
SCMS-NF-001, SCMS-NF-003 (performance, scale)	All layers + Infrastructure	Asynchronous offloading, database indexing, stateless application instances behind a load balancer
SCMS-SR-001 to SCMS-SR-007 (security)	Infrastructure, AuthModule, ComplaintModule, AuditModule	See the security architecture in section 3.9

3.9 Security Architecture

Security controls are placed at the layer that can enforce them: transport security at the edge, authentication and authorisation in the application layer, and immutability in the database. The four security objectives from SRS section 5.1.1 (SO-01 confidentiality, SO-02 integrity and non-repudiation, SO-03 least-privilege accountability, SO-04 availability and abuse resistance) drive the controls below.

3.9.1 Threat Model (STRIDE)

STRIDE category	Threat against SCMS	Control	Requirement
Spoofing	An attacker logs in as a complainant or an agent using guessed or stolen credentials	Argon2id password hashing, mobile OTP at registration, lockout after 5 failed attempts, session expiry after 30 minutes idle	SCMS-F-002, SCMS-F-005, SCMS-SR-002
Tampering	A complaint record, its status history or its evidence is altered to misrepresent what happened	Append-only audit table with no UPDATE or DELETE grant to the application account; state transitions validated by the state machine	SCMS-F-027, SCMS-SR-006
Repudiation	A staff member denies having reassigned, resolved or closed a complaint	Every state change records actor, timestamp, source IP and before and after values; reassignment requires a stored reason	SCMS-F-012, SCMS-F-027
Information disclosure	Complainant personal data or evidence files are read by someone not entitled to them	TLS 1.2+ with HSTS in transit; object-level ownership checks on every complaint operation; evidence stored outside the web root with randomised filenames; PII masked in logs	SCMS-SR-001, SCMS-SR-003, SCMS-SR-005
Denial of service	The public complaint form is flooded with automated submissions, denying service to genuine users	Rate limiting of 5 submissions per mobile number per hour and 10 login attempts per IP per 5 minutes; CAPTCHA on guest lodging; duplicate detection	SCMS-SR-007, SCMS-F-015
Elevation of privilege	A complainant or agent reaches administrative functions or another department's complaints	Deny-by-default RBAC across four roles, checked at both the view and the object level; master data and user management restricted to System Admin	SCMS-F-003, SCMS-F-024, SCMS-SR-003

3.9.2 Additional controls

- All input is validated server-side; database access uses parameterised ORM queries; output is escaped and state-changing forms carry CSRF tokens (SCMS-SR-004).
- Gateway credentials and the Django secret key are read from environment variables and are never committed to the repository.
- Session cookies are marked Secure, HttpOnly and SameSite=Lax.
- An OWASP ZAP baseline scan is part of the acceptance criteria; no High or Medium finding may remain open.

4. Design

4.1 Design Overview

Each Django app owns its models, its service functions and its views, and communicates with other apps through service functions rather than by reaching into their models. Anything that depends on a third party or on elapsed time — notification dispatch, SLA escalation, auto-closure, export generation — is a Celery task rather than part of a request. The complaint state machine lives in TrackingModule and is the single place where a transition can be approved or refused, so no other module can move a complaint into an invalid state.

4.2 UML Sequence Diagrams

Two flows are shown: the one that touches the most modules on the request path, and the one that proves the system acts without a user present.

4.2.1 SD-01 — Lodge complaint, auto-route and acknowledge

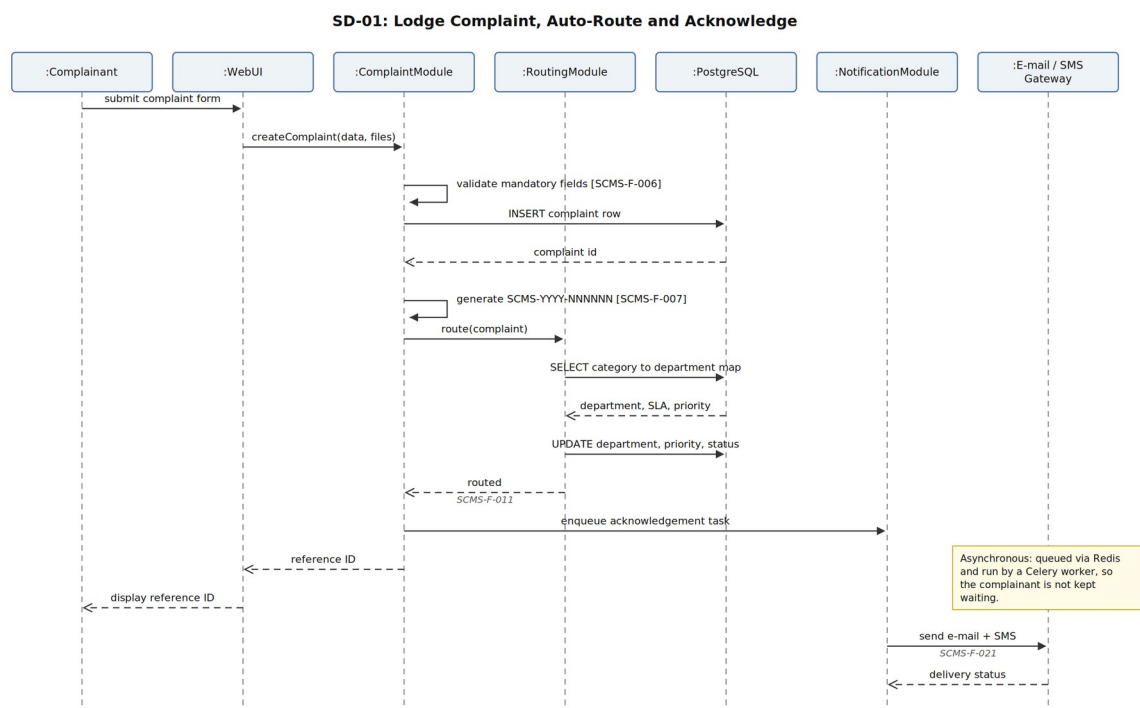


Figure 2: SD-01. The complainant receives the reference ID as soon as the complaint is stored and routed; notification dispatch continues asynchronously.

This flow realises SCMS-F-006, SCMS-F-007, SCMS-F-011 and SCMS-F-021. The key design point is the split after routing: the reference ID is returned immediately, while the acknowledgement e-mail and SMS are queued. A gateway outage therefore delays a notification but never prevents a complaint from being lodged.

4.2.2 SD-02 — SLA breach detection and escalation

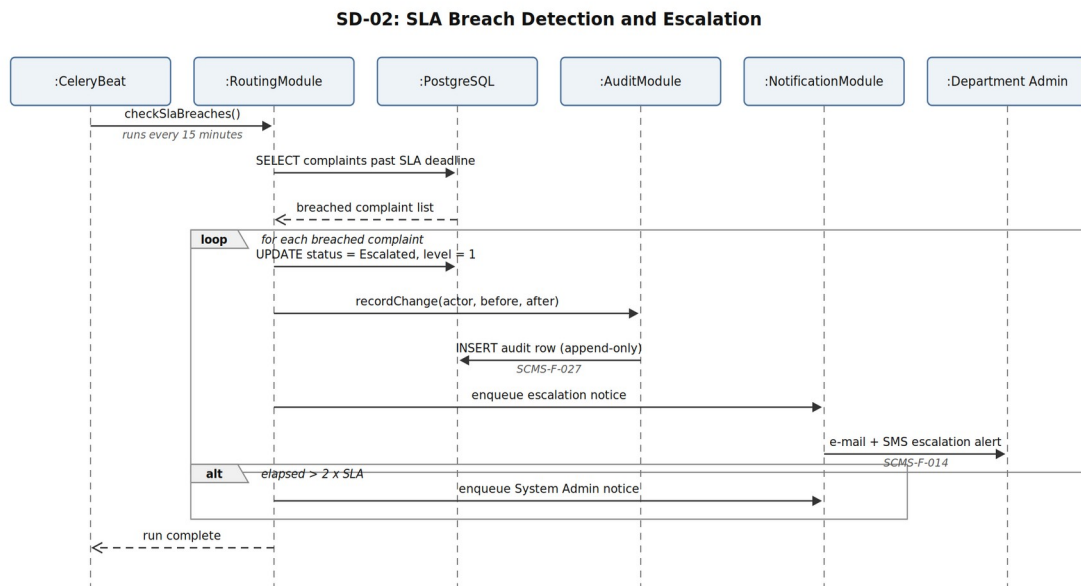


Figure 3: SD-02. Celery Beat drives escalation on a schedule; a second-level escalation to the System Admin fires at twice the SLA duration.

This flow realises SCMS-F-014 and SCMS-F-027. No human action starts it, which is why the SLA Scheduler appears as an actor in use-case diagram UC-02 of the SRS. Every escalation writes an audit row before the notification is queued, so the record exists even if dispatch later fails.

4.3 API Design

All endpoints are served over HTTPS under /api/v1/, accept and return JSON encoded in UTF-8, and require an authenticated session except where stated. Standard error codes are listed in section 4.4.

4.3.1 Complaint API (ComplaintModule)

Lodge a complaint

Endpoint : POST /api/v1/complaints
Auth : session or guest token (guest requires a verified OTP)
Request : { category_id, subcategory_id, subject, description, location, attachments[] }
Response : 201 { reference_id, status, department, priority, created_at }
Errors : 400 validation failed, 401 not authenticated, 413 attachment too large, 415 unsupported file type, 429 submission rate limit exceeded

Track a complaint

Endpoint : GET /api/v1/complaints/{reference_id}
Auth : owner, assigned agent, or admin role
Response : 200 { reference_id, subject, status, priority, department, sla_due_at, history[], comments[], attachments[] }
Errors : 403 not permitted for this complaint, 404 unknown reference

Add a comment

Endpoint : POST /api/v1/complaints/{reference_id}/comments
Request : { body }
Response : 201 { comment_id, author, body, created_at }
Errors : 403 not a participant, 409 complaint already closed

4.3.2 Assignment API (RoutingModule)

Assign or reassign to an agent

Endpoint : POST /api/v1/complaints/{reference_id}/assignment
Auth : Department Admin of the owning department
Request : { agent_id, reason }
Response : 200 { reference_id, assigned_to, status, assigned_at }
Errors : 400 reason missing on reassignment, 403 wrong department, 404 unknown agent, 409 complaint already closed

Override priority

Endpoint : PATCH /api/v1/complaints/{reference_id}/priority
Request : { priority, reason }
Response : 200 { reference_id, priority, previous_priority }
Errors : 400 invalid priority or missing reason, 403 not permitted

Escalate (internal, callable by Celery Beat)

Endpoint : POST /api/v1/complaints/{reference_id}/escalation
Request : { level, trigger } trigger = sla_breach | manual
Response : 200 { reference_id, level, escalated_to, escalated_at }
Errors : 409 already escalated at this level

4.4 Error Handling, Logging & Monitoring

Error handling

- Every API error returns a consistent body: an error code, a human-readable message, and a field map where validation failed.
- Messages shown to users never expose stack traces, SQL, file paths or internal identifiers.
- A failed login returns the same message whether the e-mail exists or not, so accounts cannot be enumerated.

- Database writes that span complaint, assignment and audit run in a single transaction, so a partial state cannot be committed.

Logging

- Structured JSON logs with a request ID carried through to any Celery task started by that request.
- Passwords, OTPs, session tokens and gateway credentials are never logged; mobile numbers and e-mail addresses are masked.
- The audit trail is separate from application logs: logs are for diagnosis and rotate, audit rows are evidence and are retained for three years.

Monitoring

Signal	Why it matters	Alert condition
95th percentile response time	Directly measures SCMS-NF-001	Above 2 seconds for 5 consecutive minutes
Application uptime	Measures SCMS-NF-002	Below 99.5% over a rolling month
Celery queue depth and Beat heartbeat	A stalled scheduler means escalations silently stop	No Beat heartbeat for 20 minutes, or queue depth above 500
Notification failure rate	Complainants being left uninformed	More than 5% permanent failures in an hour
SLA breach count by department	Operational health of the complaint process	Reported daily; not an alert
Rate-limit rejections	Early signal of spam or abuse on the public form	Above 100 rejections per hour
Failed authorisation attempts (HTTP 403)	Possible probing for other users' complaints	Above 20 from one account or IP in 10 minutes

4.5 UX Design

- Navigation adapts to the signed-in role, so an agent never sees administrative screens and a complainant never sees a queue.
- The lodging form is the shortest path in the product: category, subject, description, optional location and attachments, on one screen.
- The reference ID is shown prominently on submission and repeated in the acknowledgement message, because it is the complainant's only handle on the complaint.
- Complaint status is presented as a dated timeline rather than a single word, so progress is visible rather than implied.
- Validation messages appear beside the field concerned, never as a single summary at the top of the form.
- Every screen is operable by keyboard alone and meets WCAG 2.1 Level AA contrast; layouts work from 320 px upward (SCMS-NF-004).
- The interface supports English plus one regional language, with complaint text stored as UTF-8 so regional script is preserved.

4.6 Open Issues & Next Steps

ID	Open issue	Proposed next step
OI-01	The duplicate-detection similarity threshold of 70% is an assumption and has not been tested against real complaint text	Tune against a sample during the test phase; expose the threshold as master data rather than a constant
OI-02	The SMS gateway provider has not been selected	Select during implementation; keep the gateway behind a single adapter interface so the choice can change without touching NotificationModule
OI-03	Object store is local filesystem in development but S3-compatible in deployment	Use Django storage backends so the switch is configuration only; confirm backup consistency between files and database

ID	Open issue	Proposed next step
OI-04	No decision yet on whether regional-language support covers notification templates as well as the UI	Decide before the notification templates are written; it affects the template structure
OI-05	Malware scanning of uploads (SCMS-SR-005) requires a scanner that may not be available in the student deployment	Evaluate ClamAV in the container; if not feasible, record the gap explicitly in the test plan rather than leaving the requirement silently unmet

Next steps: freeze the schema and write the migrations, implement AuthModule and ComplaintModule first since every other module depends on them, then RoutingModule and TrackingModule, and finally reporting and audit. The test plan due on 02-10-2026 will take its cases from the acceptance criteria in SRS section 6 and the interfaces in section 4.3 above.

5. Appendices

5.1 Glossary

See section 1.4. Additional terms used in this document: ADR (Architecture Decision Record), STRIDE (threat classification model), pg_trgm (PostgreSQL trigram similarity extension), Celery Beat (periodic task scheduler), HSTS (HTTP Strict Transport Security), CSRF (Cross-Site Request Forgery).

5.2 References

- ISO/IEC/IEEE 42010:2011 — Systems and software engineering, architecture description.
- OWASP Top 10 (2021) and the OWASP Application Security Verification Standard.
- Microsoft STRIDE threat modelling methodology.
- NIST SP 800-160 — Systems Security Engineering.
- Django 5.x documentation; Django REST Framework documentation; Celery documentation.
- Digital Personal Data Protection Act, 2023 (India).
- Web Content Accessibility Guidelines (WCAG) 2.1, Level AA.

5.3 Tools

Tool	Used for
draw.io	Component and sequence diagrams in this document, and the use-case diagrams in the SRS
Jira (project T8)	Backlog of 7 epics and 42 stories and tasks, each labelled with its SRS requirement ID
GitHub	Source code and documentation repository for the mini project
pytest + coverage	Unit testing and the 70% line-coverage target (SCMS-NF-005)
flake8	PEP 8 conformance
OWASP ZAP	Baseline security scan against the security requirements
JMeter	Load testing against SCMS-NF-001 and SCMS-NF-003
Docker	Containerised build and deployment (SCMS-NF-006)